

Research proposal

Towards Decentralized Model Persistence

September 2026

Alfa Yohannis



Outline

- 1 Problem
- 2 State of the Art
- 3 Research Questions
- 4 Method and Plan
- 5 Outcomes and Resources

Can engineering models be developed in a decentralized, immutable way?

Today a model refers to another model by a file name,
and the file can change under the reference.

Problem

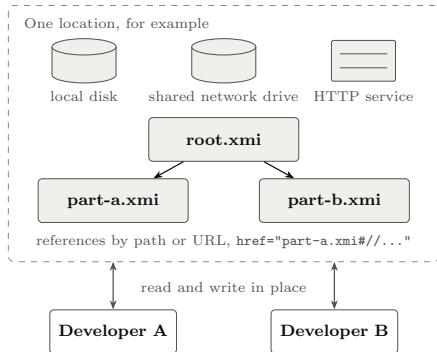
Addresses Name Locations

- The Eclipse Modeling Framework (EMF) keeps each model in a resource, and a Uniform Resource Identifier (URI) names the resource.
- Common URIs such as `file://` and `platform://` name a location, and the content at the location can change.
- A reference therefore resolves to whatever the location holds at the moment of the read.
- The InterPlanetary File System (IPFS) names content by a content identifier (CID), computed from the content itself.

A changed model receives a new CID, so a reference by CID keeps pointing at the version that was referenced.

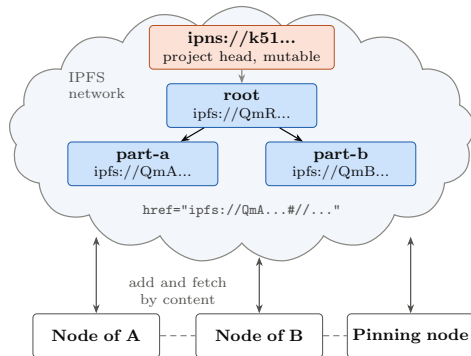
Local and Decentralized Development

(a) Local multi-fragment development



A path or URL names a location. An edit in place changes what every reader sees, and every reader needs that location.

(b) Decentralized development



A content identifier names a version. The blocks live somewhere in the network, and a reader fetches a fragment from whichever node holds a copy.

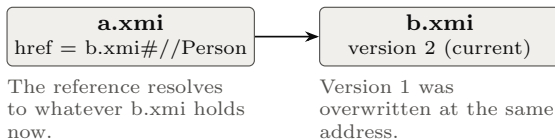
Where the Gap Hurts

- A team shares one metamodel. An edit to the shared file changes the meaning of every referring model, and no reference records the intended version.
- Reverse engineering produces a snapshot of a code base, yet a file path does not name a snapshot.
- Partners exchange models across organizations, and a path inside one organization means nothing inside another.

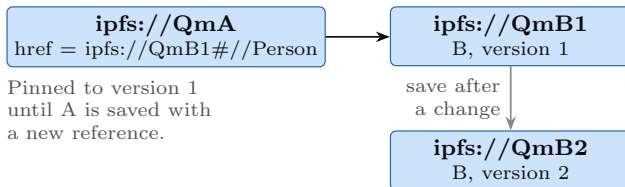
A reference that names a version would remove the ambiguity in all three settings.

Two Kinds of Address

(a) Location-based persistence



(b) Content-addressed persistence



State of the Art

What Exists and What Is Missing

- Scalable persistence, for example Morsa, NeoEMF, CDO and Hawk, keeps identifiers stable inside a store while the content changes.
- Versioning and comparison, for example EMFStore and EMF Compare, record or compute what changed between two states.
- Content addressing outside modeling, for example Venti, Git, Nix, Software Heritage and IPFS, gives addresses that never change.

No reviewed approach stores EMF models under content identifiers, and none describes references between resources whose identifiers change on every save.

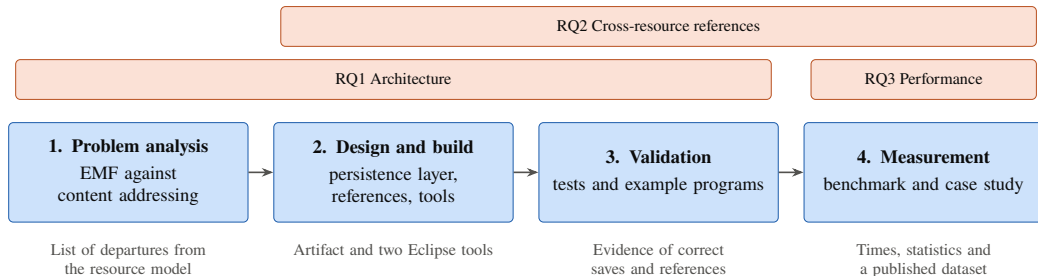
Research Questions

Three Questions

- RQ1 Architecture.** Which challenges arise when a model persistence framework stores models by content, and how can the challenges be addressed?
URI handling, a resource without an identifier before the first save, and file resources next to IPFS resources.
- RQ2 Cross-resource references.** How do references survive when every save gives a resource a new identifier?
Reference strategies, the update of referencing resources, cycles, and models split into many resources.
- RQ3 Performance.** What does content-addressed persistence cost, and when is the cost acceptable?
Save and load times against local files, from small models to several hundred thousand elements.

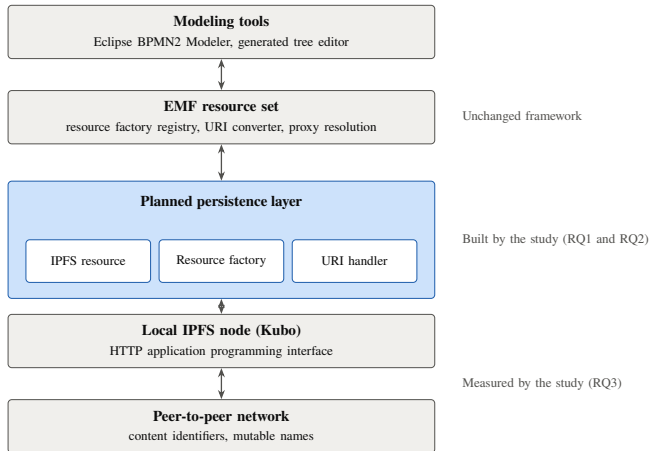
Method and Plan

Research Design

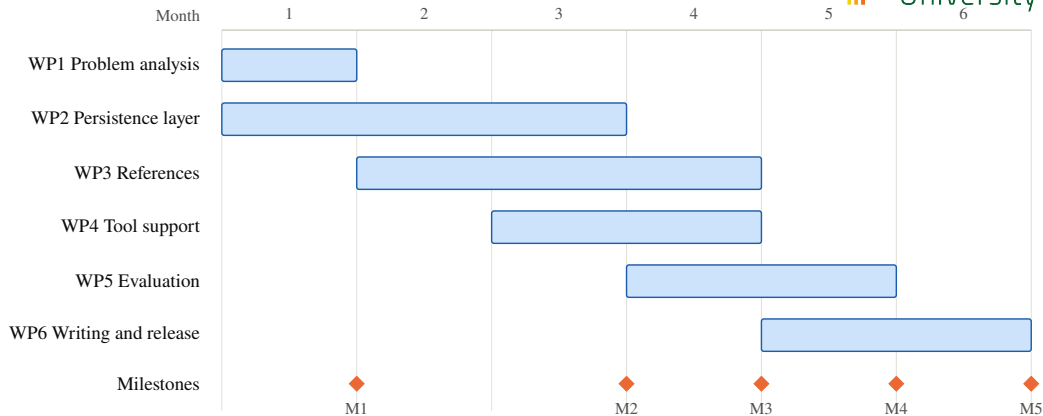


The study follows design science (Hevner et al., 2004), and the case study follows the guidelines of Runeson and Höst (2009).

Planned Persistence Layer



Work Plan



Six months, six work packages and five milestones, from the problem analysis to a submitted manuscript with a public artifact.

Evaluation Plan

Functional validation

Tests and example programs for round trips, mixed resource sets, references across resources and proxy resolution.

Benchmark

Generated models from 100 to several hundred thousand elements, thirty measured runs per size and a paired statistical test.

Case study

About one gigabyte of MoDisco models of the Eclipse Java development tools, published on IPFS and fetched back for comparison.

Outcomes and Resources

Contributions and Deliverables

- An EMF resource implementation that saves models on IPFS and loads models by CID or by a mutable IPNS name.
- Strategies for references between content-addressed resources, with an algorithm that updates referencing resources after a change.
- IPFS support inside the Eclipse BPMN2 Modeler and a generated tree editor.
- An evaluation on generated models and on large reverse-engineered models.

Deliverables: public repositories, evaluation scripts and results, a dataset archived under a DOI, and a manuscript for Software and Systems Modeling (SoSyM).

Risks and Mitigation

Risk	Mitigation
The resource model of EMF resists a changing identifier	Early proof of concept, with a placeholder address before the first save
A reference cycle cannot settle under immutable identifiers	Detect cycles before publication, and report the limit
The update of referencing resources grows expensive on a large graph	Measure on the case study model, and compare a scan with a manifest index
Measurements vary with the machine and the node	Fixed heap, warm-up runs, an idle machine, an offline node and a paired test
A published model becomes unavailable	Pin the content on a project node, and archive the dataset under a DOI

Budget

Category	USD	IDR
Researcher, 6 months at IDR 2,000,000	706	12,000,000
Assistant, 6 months at IDR 2,000,000	706	12,000,000
Storage and pinning	250	4,250,000
Two domain names at Cloudflare	22	374,000
Archiving on Zenodo	0	0
Flight to a conference in Japan or Korea	600	10,200,000
Accommodation, four nights	360	6,120,000
Conference registration	600	10,200,000
Meals and local transport, five days	250	4,250,000
Subtotal	3,494	59,394,000
Open access fee of the journal, optional	3,190	54,230,000
Total	6,684	113,624,000

Estimates at IDR 17,000 per USD. The journal fee applies only to the open access route.

Thank you

Alfa Yohannis, Department of Informatics, Pradita University
alfa.ryano@pradita.ac.id

Target venue: Software and Systems Modeling (SoSyM)